

Introduction to Artificial Intelligence

Search II

Matteo Acclavio

26/02/2026

Plan

So far:

- We have seen how to describe an agent and its environment
- We have seen how agents can plan in a 'simple' environment

Today:

- We informed planning, and plan in complex environments

Problem Formulation (and Goal definition)

- *set of states* S
- *initial state* $s_0 \in S$
- *goal states* $G \subseteq S$
- *actions* $\mathcal{A}$
- *available actions* $A : S \rightarrow \mathcal{A}(-) \subset \mathcal{A}$
- *transition model* $T : S \times \mathcal{A} \rightarrow S$
- *action cost function* $C : S \times \mathcal{A} \times S \rightarrow \mathbb{R}^+$

A **path** is a sequence of (states and) actions.

A **solution** is a path starting from the initial state and ending in a goal.

Prelude:

Tree vs Graph Search

Trees vs Graphs

Tree search

- The search space is organized into a tree;
- No tracking of visited states is needed;
- Uses less memory;
- Cannot detect loops.

Trees vs Graphs

Tree search

- The search space is organized into a tree;
- No tracking of visited states is needed;
- Uses less memory;
- Cannot detect loops.

Graph search

- The search space is organized into a graph;
- Tracking of visited states [explored set] is needed to avoid loops;
- Uses more memory;
- Can detect (and avoid) loops.

Note: both searches construct trees to explore the search space

Informed Search

Heuristic

Informed search algorithms use a *heuristic function* to estimate the cost of reaching the goal from a given state.

Idea: heuristics estimate the 'distance' to the goal.

[We use it in combination with the current path cost]

Heuristic

Informed search algorithms use a *heuristic function* to estimate the cost of reaching the goal from a given state.

Idea: heuristics estimate the 'distance' to the goal.

[We use it in combination with the current path cost]

Examples:

- For vacuum cleaner: number of dirty squares.
- For water jug problem: difference between the water in any jug and 4.
- For $3n + 1$ problem: the number of times n can divide by 2.
- For route finding: straight-line distance.

Greedy Best-First Search

Easy idea: go for the best

Greedy Best-First Search

Easy idea: go for the best

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by heuristic value, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   end if
7: end while
```

Greedy Best-First Search

Easy idea: go for the best

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by heuristic value, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   end if
7: end while
```

Note: complexity in the worst case is the same as breadth-first search, but in practice it can be much better depending on the quality of the heuristic.

A* Search

Idea: go for the best, but also consider the cost of the path so far

Pick the next node to expand based on

$$f(n) = \underbrace{g(n)}_{\text{path cost up to } n} + \underbrace{h(n)}_{\text{estimated cost to reach a goal from } n}$$

A* Pseudocode

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost + heuristic value, and node ←
   frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

A* Pseudocode

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost + heuristic value, and node ←
   frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

Note: complexity in the worst case is the same as breadth-first search, but in practice it can be much better depending on the quality of the heuristic.

Evaluating heuristics

How do we evaluate if a heuristic is good?

Evaluating heuristics

How do we evaluate if a heuristic is good?

- ***Admissible heuristic***: never overestimates the cost to reach the goal.

Evaluating heuristics

How do we evaluate if a heuristic is good?

- ***Admissible heuristic***: never overestimates the cost to reach the goal.

$$h(n) \leq c(n)$$

where $c(n)$ is the actual cost from n to a goal.

Evaluating heuristics

How do we evaluate if a heuristic is good?

- **Admissible heuristic**: never overestimates the cost to reach the goal.

$$h(n) \leq c(n)$$

where $c(n)$ is the actual cost from n to a goal.

- **Consistent heuristic** [triangular inequality]: for every node n and every successor n' of n , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' .

Evaluating heuristics

How do we evaluate if a heuristic is good?

- **Admissible heuristic**: never overestimates the cost to reach the goal.

$$h(n) \leq c(n)$$

where $c(n)$ is the actual cost from n to a goal.

- **Consistent heuristic** [triangular inequality]: for every node n and every successor n' of n , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' .

$$h(n) \leq c(n, n') + h(n')$$

where $c(n, n')$ is the cost of the action that leads from n to n' .

Admissible heuristic are good for tree search

Theorem

If the heuristic is admissible, then A^ tree search is optimal.*

Proof.

BLACKBOARD



Consistent heuristics are good for graph search

Theorem

If the heuristic is consistent, then A^ graph search is optimal.*

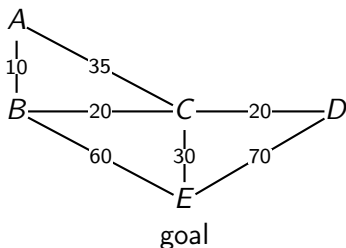
Proof.

BLACKBOARD



Example admissibility and consistency

start



$$h(A) = 60$$

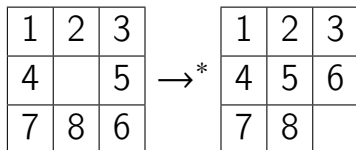
$$h(B) = 50$$

$$h(C) = 20$$

$$h(D) = 60$$

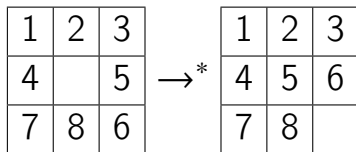
Example of heuristics: the 8-puzzle

Slide Puzzle (8-puzzle)



Example of heuristics: the 8-puzzle

Slide Puzzle (8-puzzle)

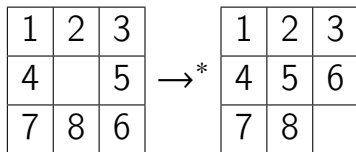


Dimension of the state space: $9! = 362,880$ states.

Note: only half of the states are reachable

Example of heuristics: the 8-puzzle

Slide Puzzle (8-puzzle)



Dimension of the state space: $9! = 362,880$ states.

Note: only half of the states are reachable

Possible heuristics:

- Number of misplaced tiles.
- Sum of the Manhattan distances of the tiles from their goal positions.

Heuristics and performance

In general:

- The more informed the heuristic, the better the performance of A^* .

Heuristics and performance

In general:

- The more informed the heuristic, the better the performance of A^* .
- More informed heuristics may be more expensive to compute.

Heuristics and performance

In general:

- The more informed the heuristic, the better the performance of A^* .
- More informed heuristics may be more expensive to compute.
- Heuristics can come from a database of previously solved problems, or from learning.

Search in Complex Environments

Search in Complex Environments

We are going to drop some of the assumptions we made so far on the environment:

- Fully observable $\rightarrow$ partially observable

Search in Complex Environments

We are going to drop some of the assumptions we made so far on the environment:

- Fully observable $\rightarrow$ partially observable
- Discrete $\rightarrow$ continuous

Search in Complex Environments

We are going to drop some of the assumptions we made so far on the environment:

- Fully observable $\rightarrow$ partially observable
- Discrete $\rightarrow$ continuous
- Deterministic $\rightarrow$ non-deterministic

Search in Complex Environments

We are going to drop some of the assumptions we made so far on the environment:

- Fully observable $\rightarrow$ partially observable
- Discrete $\rightarrow$ continuous
- Deterministic $\rightarrow$ non-deterministic

and see how to adapt our search algorithms and the way we represent solutions.

Partially Observable Environments I

Partially Observable Environments (Local information)

Local information: the agent has only access to adjacent states.

Partially Observable Environments (Local information)

Local information: the agent has only access to adjacent states.

- no need of much memory
- no need to care about the infinite state space

Partially Observable Environments (Local information)

Local information: the agent has only access to adjacent states.

- no need of much memory
- no need to care about the infinite state space

Idea: no planning, but just try to improve the current state by looking at the neighbors.

Partially Observable Environments (Local information)

Local information: the agent has only access to adjacent states.

- no need of much memory
- no need to care about the infinite state space

Idea: no planning, but just try to improve the current state by looking at the neighbors.

Note: the agent may not explore the entire space [local solutions]

Hill climbing [Greedy local search]

Goal: find a local maximum of the heuristic function.

How:

Hill climbing [Greedy local search]

Goal: find a local maximum of the heuristic function.

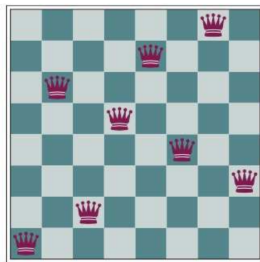
How: at each step, move to the neighbor with the highest heuristic value (if it is better than the current state).

8-queen problem

State: any configuration of 8 queens on the chessboard.

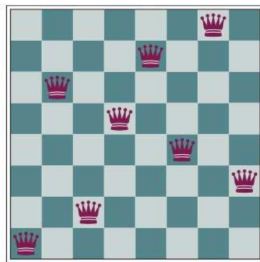
8-queen problem

State: any configuration of 8 queens on the chessboard. **Goal:** put 8 queens on a chessboard such that no two queens threaten each other.



8-queen problem

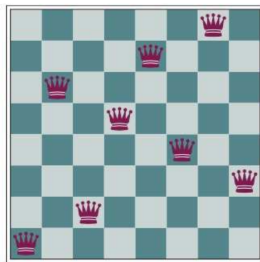
State: any configuration of 8 queens on the chessboard. **Goal:** put 8 queens on a chessboard such that no two queens threaten each other.



Size of the state space: $\binom{64}{8} = 7.392.009.768$.

8-queen problem

State: any configuration of 8 queens on the chessboard. **Goal:** put 8 queens on a chessboard such that no two queens threaten each other.

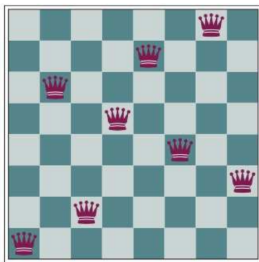


Size of the state space: $\binom{64}{8} = 7.392.009.768$.

Size of neighborhood: $8 \times 7 = 56$.

8-queen problem

State: any configuration of 8 queens on the chessboard. **Goal:** put 8 queens on a chessboard such that no two queens threaten each other.



Size of the state space: $\binom{64}{8} = 7.392.009.768$.

Size of neighborhood: $8 \times 7 = 56$.

Heuristic: number of pairs of queens that threaten each other [$h(\text{sol}) = 0$].

Problems with local search

- Local maxima: the agent may get stuck in a local maximum that is not a global maximum.

Problems with local search

- Local maxima: the agent may get stuck in a local maximum that is not a global maximum.
- Plateaus: the agent may get stuck in a flat area of the heuristic function where all neighbors have the same value.

Problems with local search

- Local maxima: the agent may get stuck in a local maximum that is not a global maximum.
- Plateaus: the agent may get stuck in a flat area of the heuristic function where all neighbors have the same value.

Greedy local search on 8-queen problem get stuck in local maxima and plateaus 86% of the time

Problems with local search

- Local maxima: the agent may get stuck in a local maximum that is not a global maximum.
- Plateaus: the agent may get stuck in a flat area of the heuristic function where all neighbors have the same value.

Greedy local search on 8-queen problem get stuck in local maxima and plateaus 86% of the time ... but it is very fast to reach a local maximum

[starting from random initial state, average 4 steps on average when successful, average 3 steps when failed]

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)
- Simulated annealing: allow moves to worse neighbors with a probability that decreases over time

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)
- Simulated annealing: allow moves to worse neighbors with a probability that decreases over time
- Local beam search: keep track of the k best states at each step, and expand them all

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)
- Simulated annealing: allow moves to worse neighbors with a probability that decreases over time
- Local beam search: keep track of the k best states at each step, and expand them all
- Genetic algorithms: maintain a population of states, and use selection, crossover, and mutation to generate new states

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)
- Simulated annealing: allow moves to worse neighbors with a probability that decreases over time
- Local beam search: keep track of the k best states at each step, and expand them all
- Genetic algorithms: maintain a population of states, and use selection, crossover, and mutation to generate new states

Escaping local maxima and plateaus

Possible solutions:

- Random restart: run the algorithm multiple times from different random initial states
- Allow 'sideways moves' to moves to neighbors with the same heuristic value (for a bound number of steps)
- Simulated annealing: allow moves to worse neighbors with a probability that decreases over time
- Local beam search: keep track of the k best states at each step, and expand them all
- Genetic algorithms: maintain a population of states, and use selection, crossover, and mutation to generate new states

Continuous Environments

Search in Continuous State Spaces

Example:

- A drone flying in the sky

Search in Continuous State Spaces

Example:

- A drone flying in the sky
- Position an object 'at the center' of a set of points

Search in Continuous State Spaces

Example:

- A drone flying in the sky
- Position an object 'at the center' of a set of points
- A stock-market bot trying to maximize the profit

Search in Continuous State Spaces

Example:

- A drone flying in the sky
- Position an object 'at the center' of a set of points
- A stock-market bot trying to maximize the profit

Objective function is a function $f: \mathbb{R}^n \mapsto \mathbb{R}$

Local Search in Continuous State Spaces

Discretization: fix 'units' to build a 'grid' the state space, and apply search algorithms on the resulting discrete state space.

Local Search in Continuous State Spaces

Discretization: fix 'units' to build a 'grid' the state space, and apply search algorithms on the resulting discrete state space.

Gradient method: Look at the 'slope' in any 'direction' of the space, and move in the steepest direction.

Local Search in Continuous State Spaces

Discretization: fix 'units' to build a 'grid' the state space, and apply search algorithms on the resulting discrete state space.

Gradient method: Look at the 'slope' in any 'direction' of the space, and move in the steepest direction.

Newton-Raphson method: Look not only to the derivative (slope) but also to the second derivative (curvature) of the function, and move accordingly.

Local Search in Continuous State Spaces

Discretization: fix 'units' to build a 'grid' the state space, and apply search algorithms on the resulting discrete state space.

Gradient method: Look at the 'slope' in any 'direction' of the space, and move in the steepest direction.

Newton-Raphson method: Look not only to the derivative (slope) but also to the second derivative (curvature) of the function, and move accordingly.

Linear programming: If the objective function is linear.

Non-deterministic Environments

Problem: actions may have multiple possible outcomes, and the agent does not control which one occurs.

Idea: the agent has a 'belief state', keeping track of all the possible states it could be in

Erratic Vacuum Cleaner

Example with our vacuum cleaner:

- the floor is slippery, and some moves may fail

Erratic Vacuum Cleaner

Example with our vacuum cleaner:

- the floor is slippery, and some moves may fail
- while cleaning a room, the agent may accidentally make the floor dirty again

Erratic Vacuum Cleaner

Example with our vacuum cleaner:

- the floor is slippery, and some moves may fail
- while cleaning a room, the agent may accidentally make the floor dirty again
- while cleaning a room, the agent may accidentally clean the other room

How to plan?

- Construct an and-or tree to represent the search space:
 - **or** nodes represent the choice of actions to take
 - **and** nodes represent the possible outcomes of actions
- Use if – then – else construct to plan for all possible outcomes of actions
- Use while construct to plan for possible loops in the search space

Partially Observable Environments II

Limited or no observability

Problem: the agent has no access to the current state or only partial access to it

Limited or no observability

Problem: the agent has no access to the current state or only partial access to it

Idea: the agent has a 'belief state' keeping track of all the possible states it could be in, and update it based on the actions taken and the observations received.

Maze example

Problem: agent in a maze with no access to its current position, but it can sense the walls around it.

Maze example

Problem: agent in a maze with no access to its current position, but it can sense the walls around it.

Solution: proceed by always keeping the wall on the left side, and turn left whenever possible

Blind vacuum cleaner example

Vacuum cleaner with no access to its current position

Online vs Offline Search

Online vs Offline Search

- **Offline search:** the agent has access plans a complete solution before acting, no update of the plan is possible during execution.

Online vs Offline Search

- **Offline search:** the agent has access plans a complete solution before acting, no update of the plan is possible during execution.
- **Online search:** the agent interleaves planning and execution, and can update its plan based on the observations received during execution.

Summary

Book references:

- Chapter 3 [all]
- Chapter 4.1 [Local search]
- Chapter 4.2 [Search in continuous state spaces]
- Chapter 4.3 [Search in non-deterministic environments]
- Chapter 4.4 [Search in partially observable environments]
- Chapter 4.5 [Online search]

Next TA session:

- Exercises on Search [Chapter 3 and 4 so far]

Next lecture:

- What we did not cover today
- Factorizing the state space